Bloque 6. Cursores e excepcións

1. Procesamento avanzado de datos en MariaDB

Nas unidades anteriores vimos como crear procedementos, funcións, eventos e disparadores para automatizar tarefas nunha base de datos. En moitas situacións, estas instrucións permiten operar directamente sobre conxuntos completos de rexistros mediante sentenzas SQL.

Porén, existen operacións nas que necesitamos procesar os resultados dunha consulta fila a fila. Para iso utilízanse os **cursores**.

Ademais, durante a execución dun programa almacenado poden producirse erros ou situacións especiais, como tentar obter unha fila que xa non existe. MariaDB ofrece mecanismos para controlar estas situacións mediante **controladores de condicións e excepcións**.

- **Cursores:** permiten percorrer os resultados dunha consulta e procesar cada fila individualmente.
- **Controladores (HANDLER):** permiten definir como debe reaccionar un programa cando se produce un erro ou unha condición determinada.
- **SIGNAL e RESIGNAL:** permiten xerar erros personalizados ou volver lanzar unha condición.

2. Cursores en MariaDB

2.1. Concepto de cursor

Un cursor é un mecanismo que permite acceder de maneira secuencial ás filas devolvidas por unha consulta SQL. En lugar de traballar con todos os resultados á vez, o cursor permite obter unha fila, procesala e continuar coa seguinte.

O seu funcionamento é similar ao percorrido dunha lista de elementos:

1. Defínese a consulta que proporcionará os rexistros.
2. Declárase o cursor asociado a esa consulta.
3. Ábrese o cursor para executar a consulta.
4. Recupéranse as filas unha a unha.
5. Procésase cada fila mediante instrucións SQL.
6. Péchase o cursor cando remata o percorrido.

> **Importante:** os cursores en MariaDB utilízanse normalmente dentro de procedementos almacenados, funcións ou bloques de código que permiten declarar variables e controlar o fluxo de execución.

2.2. Declaración dun cursor: DECLARE CURSOR

Para crear un cursor utilizamos `DECLARE CURSOR`, indicando a consulta SQL que devolverá os rexistros que queremos percorrer.

```
DECLARE nome_cursor CURSOR FOR
    SELECT columnas
    FROM nome_taboa
    WHERE condicion;
```

Por exemplo, podemos declarar un cursor para percorrer os alumnos dun curso:

```
DECLARE cursor_alumnos CURSOR FOR
    SELECT id, nome
    FROM alumnos
    WHERE curso_id = 1;
```

A declaración non executa aínda a consulta nin recupera os datos. Só define o cursor e a consulta asociada.

2.3. Apertura do cursor: OPEN

Para executar a consulta e preparar os resultados utilizamos `OPEN`.

```
OPEN cursor_alumnos;
```

Unha vez aberto, o cursor queda preparado para recuperar as filas devolvidas pola consulta.

2.4. Lectura dunha fila: FETCH

A instrución `FETCH` permite recuperar a seguinte fila do cursor e almacenar os valores nas variables indicadas.

```
FETCH cursor_alumnos INTO v_id, v_nome;
```

As variables deben declararse previamente e ser compatibles cos tipos de datos das columnas recuperadas.

Cada execución de `FETCH` avanza o cursor ata a seguinte fila. Cando non quedan máis filas, MariaDB produce a condición `NOT FOUND`.

2.5. Peche do cursor: CLOSE

Cando remata o procesamento dos resultados debemos pechar o cursor.

```
CLOSE cursor_alumnos;
```

O peche libera os recursos asociados ao cursor. Un cursor aberto debe pecharse cando xa non sexa necesario utilizalo.

2.6. Resumo das operacións

Instrución	Función
`DECLARE CURSOR`	Define o cursor e a consulta que vai executar.
`OPEN`	Abre o cursor e prepara os resultados.
`FETCH`	Recupera a seguinte fila do resultado.
`CLOSE`	Pech a o cursor e libera os recursos.

3. Procesamento fila a fila

3.1. Necesidade de estruturas repetitivas

Como `FETCH` recupera unha fila cada vez, necesitamos unha estrutura repetitiva para percorrer todos os resultados do cursor.

Podemos utilizar estruturas como:

- `LOOP`: repite un bloque de instrucións ata que se executa unha saída explícita.

- `WHILE` : repite mentres se cumpra unha condición.
- `REPEAT` : executa o bloque polo menos unha vez e repíteo ata que se cumpra unha condición.

Un patrón habitual consiste en utilizar un bucle `LOOP` e un controlador para detectar cando o cursor xa non ten máis filas.

3.2. Variables locais

As variables locais permiten almacenar temporalmente os valores recuperados polo cursor e realizar operacións con eles.

Decláranse mediante `DECLARE` ao comezo do bloque no que se utilizan.

```
DECLARE v_id INT;
DECLARE v_nome VARCHAR(100);
DECLARE v_total INT DEFAULT 0;
```

Podemos utilizar estas variables para calcular totais, comprobar condicións ou inserir información noutras táboas.

3.3. Exemplo completo: percorrer alumnos

O seguinte procedemento percorre os alumnos dun curso e mostra os seus identificadores e nomes mediante mensaxes de saída.

```
DELIMITER //

CREATE PROCEDURE percorrer_alumnos()
BEGIN
    DECLARE fin BOOLEAN DEFAULT FALSE;
    DECLARE v_id INT;
    DECLARE v_nome VARCHAR(100);

    DECLARE cursor_alumnos CURSOR FOR
        SELECT id, nome
        FROM alumnos;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET fin = TRUE;

    OPEN cursor_alumnos;

    bucle: LOOP

        FETCH cursor_alumnos INTO v_id, v_nome;

        IF fin THEN
            LEAVE bucle;
        END IF;

        SELECT v_id AS id_alumno,
               v_nome AS nome_alumno;

    END LOOP;

    CLOSE cursor_alumnos;
END //

DELIMITER ;
```

Para executar o procedemento:

```
CALL percorrer_alumnos();
```

O procedemento realiza as seguintes operacións:

1. Declara unha variable de control chamada `fin`.
2. Declara as variables que almacenarán os datos de cada alumno.
3. Define o cursor coa consulta que selecciona os alumnos.
4. Declara un controlador para detectar o final dos resultados.

5. Abre o cursor.
6. Recupera unha fila en cada iteración mediante `FETCH`.
7. Se non quedan filas, abandona o bucle mediante `LEAVE`.
8. Procesa os datos recuperados.
9. Péchase o cursor ao finalizar.

3.4. Exemplo: calcular un total mediante un cursor

Un cursor tamén pode utilizarse para realizar cálculos que dependan do procesamento individual de cada rexistro.

O seguinte procedemento percorre os produtos e calcula a suma dos seus stocks.

```
DELIMITER //

CREATE PROCEDURE calcular_stock_total(
    OUT p_total INT
)
BEGIN
    DECLARE fin BOOLEAN DEFAULT FALSE;
    DECLARE v_stock INT;

    DECLARE cursor_stock CURSOR FOR
        SELECT stock
        FROM produtos;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET fin = TRUE;

    SET p_total = 0;

    OPEN cursor_stock;

    bucle: LOOP

        FETCH cursor_stock INTO v_stock;

        IF fin THEN
            LEAVE bucle;
        END IF;

        SET p_total = p_total + v_stock;

    END LOOP;

    CLOSE cursor_stock;
END //

DELIMITER ;
```

Para executar o procedemento e consultar o resultado:

```
CALL calcular_stock_total(@total);

SELECT @total AS stock_total;
```

Neste exemplo, o cursor recupera o stock de cada produto e acumula os valores na variable de saída `p_total`.

> **Boas prácticas:** cando unha operación pode resolverse cunha única instrución SQL, normalmente é preferible utilizala en lugar dun cursor. As operacións fila a fila adoitan ser menos eficientes que as operacións sobre conxuntos completos de rexistros.

4. Declaracións dentro dun bloque almacenado

En MariaDB é importante respectar a orde das declaracións dentro dun bloque `BEGIN ... END`.

As variables, cursores e controladores deben declararse antes de executar as instrucións do bloque.

A orde correcta é:

1. Variables locais e condicións declaradas mediante `DECLARE`.
2. Cursores declarados mediante `DECLARE ... CURSOR`.
3. Controladores declarados mediante `DECLARE ... HANDLER`.
4. Instrucións executables: `SET`, `OPEN`, `FETCH`, bucles, condicións, etc.

```
BEGIN
    -- 1. Variables
    DECLARE fin BOOLEAN DEFAULT FALSE;
    DECLARE v_id INT;

    -- 2. Cursor
    DECLARE cursor_datos CURSOR FOR
        SELECT id FROM produtos;

    -- 3. Controlador
    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET fin = TRUE;

    -- 4. Instrucións executables
    OPEN cursor_datos;

    -- Procesamento...

    CLOSE cursor_datos;
END
```

> **Importante:** non se poden declarar variables, cursores ou controladores despois de comezar a executar as instrucións do bloque. Unha declaración situada nunha posición incorrecta provocará un erro de sintaxe.

5. Tratamento de excepcións e condicións

5.1. Concepto de excepción

Durante a execución dun procedemento ou función poden producirse situacións que impidan continuar normalmente coa operación.

Algúns exemplos son:

- Intentar inserir un valor duplicado nunha columna cunha restrición `UNIQUE`.
- Inserir un valor que incumpre unha chave allea.
- Recuperar un rexistro que non existe.
- Dividir un valor entre cero.
- Executar unha operación que non cumpre as condicións definidas polo programa.

O tratamento de excepcións permite controlar estas situacións e decidir que debe facer o programa cando se produce un erro ou unha condición especial.

5.2. Controladores mediante DECLARE HANDLER

MariaDB utiliza a instrución `DECLARE HANDLER` para definir como debe reaccionar un bloque cando se produce unha condición determinada.

A súa sintaxe xeral é:

```
DECLARE tipo_handler HANDLER
FOR condicion
instrucion;
```

Os elementos principais son:

- **Tipo de controlador:** indica se o programa continúa ou sae do bloque.
- **Condición:** identifica o erro ou situación que activa o controlador.
- **Instrución:** acción que se executará cando se produza a condición.

5.3. Tipos de controladores: CONTINUE e EXIT

MariaDB ofrece dous tipos principais de controladores:

Tipo	Funcionamento
`CONTINUE`	Executa a instrución do controlador e continúa coa execución desde o punto posterior ao que provocou a condición.
`EXIT`	Executa a instrución do controlador e abandona o bloque `BEGIN ... END` no que foi declarado.

5.4. Exemplo de CONTINUE

O seguinte exemplo controla unha condición de división entre cero. Cando se produce o erro, o controlador asigna un valor alternativo á variable e permite continuar.

```
DELIMITER //

CREATE PROCEDURE exemplo_continue()
BEGIN
    DECLARE resultado DECIMAL(10,2);
    DECLARE CONTINUE HANDLER FOR SQLSTATE '22012'
        SET resultado = NULL;

    SET resultado = 10 / 0;

    SELECT resultado AS resultado;
    SELECT 'O procedemento continúa' AS mensaxe;
END //

DELIMITER ;
```

O controlador captura a condición de división entre cero e permite que o procedemento continúe coas instrucións posteriores.

5.5. Exemplo de EXIT

O controlador `EXIT` permite abandonar o bloque no que foi declarado cando se produce unha condición.

```
DELIMITER //

CREATE PROCEDURE exemplo_exit()
BEGIN
    DECLARE resultado INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Produciuse un erro na operación' AS mensaxe;
    END;

    INSERT INTO produtos(id, nome, prezo, stock)
    VALUES (1, 'Produto duplicado', 10.00, 5);

    SET resultado = 1;

    SELECT resultado;
END //

DELIMITER ;
```

Se a inserción provoca unha excepción SQL, execútase o controlador e abandónase o bloque. As instrucións posteriores á operación que provocou o erro non se executan.

6. Tipos de condicións e erros

6.1. Condicións xerais

Os controladores poden asociarse a diferentes tipos de condicións.

Condición	Significado
`SQLEXCEPTION`	Captura erros SQL, agás as condicións de clase SQLWARNING e NOT FOUND.
`SQLWARNING`	Captura condicións de advertencia.
`NOT FOUND`	Indica que non se atopou unha fila ou que non quedan máis filas nun cursor.
Código de erro	Permite identificar un erro concreto mediante o seu código numérico.
SQLSTATE	Identifica unha condición mediante un código normalizado de cinco caracteres.

6.2. Condición NOT FOUND

A condición `NOT FOUND` é especialmente importante cando traballamos con cursores.

Ao executar `FETCH`, se non quedan máis filas que recuperar, MariaDB activa esta condición. Se non existe un controlador para tratala, o programa pode finalizar cun erro ou non realizar o percorrido esperado.

Unha solución habitual é declarar unha variable de control e un controlador `CONTINUE`:

```
DECLARE fin BOOLEAN DEFAULT FALSE;

DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET fin = TRUE;
```

Cando o cursor chega ao final, a variable `fin` pasa a valer `TRUE`. O bucle pode consultar esa variable para abandonar o percorrido.

6.3. Control de erros mediante SQLSTATE

SQLSTATE é un código de cinco caracteres que identifica unha condición producida durante a execución dunha instrución SQL.

Podemos declarar un controlador para unha condición concreta:

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '23000'
    SET @erro_integridade = TRUE;
```

O código `23000` corresponde á clase de erros relacionados coa integridade das restricións, como determinadas violacións de claves ou restricións de unicidade.

6.4. Control de erros mediante códigos numéricos

Algúns erros de MariaDB poden identificarse mediante o seu código numérico.

```
DECLARE CONTINUE HANDLER FOR 1062
    SET @erro_duplicado = TRUE;
```

Neste exemplo, o controlador captura o erro 1062, asociado á duplicación dun valor nunha chave única ou primaria.

6.5. Declaración de condicións con DECLARE CONDITION

Podemos asignar un nome a unha condición para facer máis lexible o código.

```
DECLARE duplicado CONDITION FOR 1062;

DECLARE CONTINUE HANDLER FOR duplicado
    SET @erro_duplicado = TRUE;
```

O nome `duplicado` representa a condición identificada polo código de erro 1062.

7. Xeración de erros personalizados: SIGNAL

7.1. Concepto de SIGNAL

A instrución `SIGNAL` permite provocar intencionadamente unha condición de erro durante a execución dun procedemento, función ou disparador.

É útil cando queremos establecer regras propias e impedir que unha operación continúe se non cumpre determinadas condicións.

A sintaxe básica é:

```
SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Descrición do erro';
```

O código `45000` utilízase habitualmente para representar unha excepción definida polo usuario.

7.2. Exemplo de validación mediante SIGNAL

O seguinte procedemento comproba que a cantidade dun pedido sexa maior que cero.

```
DELIMITER //

CREATE PROCEDURE crear_pedido(
    IN p_id_produto INT,
    IN p_cantidade INT
)
BEGIN
    IF p_cantidade <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'A cantidade debe ser maior que cero';
    END IF;

    INSERT INTO pedidos(id_produto, cantidade)
    VALUES (p_id_produto, p_cantidade);
END //

DELIMITER ;
```

Se se chama ao procedemento cunha cantidade igual ou inferior a cero, MariaDB xera o erro personalizado e non executa a inserción.

```
CALL crear_pedido(1, -5);
```

7.3. Utilidade de SIGNAL

Esta instrución permite:

- Impedir operacións que incumpren regras de negocio.
- Devolver mensaxes de erro comprensibles.
- Validar parámetros recibidos por procedementos.
- Controlar situacións que non están representadas por un erro SQL estándar.

8. Relanzamento de excepcións: RESIGNAL

8.1. Concepto de RESIGNAL

A instrución `RESIGNAL` permite volver lanzar unha condición que xa foi capturada por un controlador.

É útil cando queremos executar algunha acción antes de que o erro se comunique ao programa que chamou ao procedemento.

Por exemplo, podemos capturar un erro, rexistrar unha mensaxe e volver lanzar a excepción.

```
DELIMITER //

CREATE PROCEDURE inserir_produto(
    IN p_id INT,
    IN p_nome VARCHAR(100)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        INSERT INTO rexistro_erro(mensaxe, data_erro)
        VALUES ('Erro ao inserir un produto', NOW());

        RESIGNAL;
    END;

    INSERT INTO produtos(id, nome)
    VALUES (p_id, p_nome);
END //

DELIMITER ;
```

Neste exemplo, se a inserción falla:

1. Actívase o controlador `EXIT`.
2. Insírese unha mensaxe na táboa de rexistro de erros.
3. `RESIGNAL` volve lanzar a excepción orixinal.
4. O programa que chamou ao procedemento recibe o erro.

> **Diferenza entre SIGNAL e RESIGNAL:**
>
> - `SIGNAL` crea unha nova condición ou erro.
> - `RESIGNAL` volve lanzar unha condición que xa foi capturada por un controlador.

9. Integración de cursores e controladores

Os cursores e os controladores utilízanse con frecuencia de maneira conxunta. O cursor permite percorrer os resultados dunha consulta e o controlador permite detectar cando remata o percorrido.

9.1. Exemplo completo

O seguinte procedemento percorre os produtos dunha táboa, calcula o valor total do stock e almacena o resultado nunha variable de saída.

```
DELIMITER //

CREATE PROCEDURE calcular_valor_inventario(
    OUT p_total DECIMAL(12,2)
)
BEGIN
    DECLARE fin BOOLEAN DEFAULT FALSE;
    DECLARE v_prezo DECIMAL(10,2);
    DECLARE v_stock INT;

    DECLARE cursor_produtos CURSOR FOR
        SELECT prezo, stock
        FROM produtos;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET fin = TRUE;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        CLOSE cursor_produtos;
        RESIGNAL;
    END;

    SET p_total = 0;

    OPEN cursor_produtos;

    bucle: LOOP

        FETCH cursor_produtos INTO v_prezo, v_stock;

        IF fin THEN
            LEAVE bucle;
        END IF;

        SET p_total = p_total + (v_prezo * v_stock);

    END LOOP;

    CLOSE cursor_produtos;
END //

DELIMITER ;
```

Para executar o procedemento:

```
CALL calcular_valor_inventario(@valor);

SELECT @valor AS valor_total_inventario;
```

O procedemento combina varios elementos:

- Un cursor para recuperar o prezo e o stock de cada produto.
- Variables locais para almacenar os valores recuperados.
- Un controlador `NOT FOUND` para detectar o final do cursor.
- Un bucle para procesar todos os produtos.
- Un controlador de excepcións para tratar erros SQL.
- Unha variable de saída para devolver o resultado.

> **Nota técnica:** nun procedemento real, o peche do cursor dentro dun controlador debe deseñarse tendo en conta se o cursor chegou a abrirse. Tamén é importante evitar que un segundo erro producido durante a xestión do erro oculte a excepción orixinal. O exemplo ilustra a combinación dos mecanismos, pero a xestión de recursos debe adaptarse ao fluxo concreto do programa.

10. Boas prácticas no uso de cursores e excepcións

- Utilizar cursores unicamente cando sexa necesario procesar os rexistros individualmente.
- Priorizar as operacións SQL sobre conxuntos de datos cando permitan resolver o problema de maneira máis eficiente.
- Declarar as variables, cursores e controladores antes das instrucións executables.
- Pechar os cursores unha vez finalizado o seu uso.
- Controlar sempre a condición `NOT FOUND` nos percorridos de cursores.
- Escoller entre `CONTINUE` e `EXIT` segundo o comportamento que deba ter o programa.
- Utilizar controladores específicos cando se coñecen os erros que poden producirse.
- Evitar ocultar os erros con controladores demasiado xenéricos.
- Utilizar `SIGNAL` para comunicar erros relacionados coas regras de negocio.
- Utilizar `RESIGNAL` cando sexa necesario propagar unha excepción despois de realizar tarefas adicionais.
- Probar os procedementos con datos válidos e con situacións que provoquen erros.

11. Exercicios prácticos

Exercicio 1. Percorrido de rexistros mediante un cursor

Traballa cunha táboa de alumnos que conteña, como mínimo, identificador, nome e curso.

1. Crea un procedemento que percorra os alumnos mediante un cursor.
2. Recupera o identificador e o nome de cada alumno.
3. Mostra os datos de cada rexistro durante o percorrido.
4. Controla o final dos resultados mediante unha condición `NOT FOUND`.
5. Comproba o funcionamento con táboas que teñan varios rexistros e tamén sen rexistros.

Exercicio 2. Cálculos mediante cursores

Utiliza unha táboa de produtos que conteña nome, prezo e stock.

1. Crea un procedemento que percorra todos os produtos.
2. Calcula o valor total do inventario.
3. Obtén o número de produtos procesados.
4. Devolve os resultados mediante parámetros de saída.
5. Compara o resultado co obtido mediante unha consulta SQL que utilice funcións de agregación.

Exercicio 3. Control de erros mediante HANDLER

Deseña varios procedementos que realicen operacións sobre unha táboa de produtos.

1. Define un controlador `CONTINUE` para unha condición de erro concreta.
2. Define outro controlador `EXIT` para capturar erros SQL.
3. Realiza operacións que provoquen os erros controlados.
4. Comproba que a execución continúa ou finaliza segundo o tipo de controlador utilizado.
5. Rexistra información sobre os erros producidos.

Exercicio 4. Validación con SIGNAL

Desenvolve un procedemento para rexistrar pedidos nunha base de datos.

1. Define os parámetros necesarios para identificar o produto e a cantidade solicitada.
2. Comproba que a cantidade sexa válida.
3. Impide rexistrar pedidos que incumpran as regras establecidas.

4. Utiliza `SIGNAL` para devolver mensaxes de erro personalizadas.
5. Proba o procedemento con diferentes valores.

Exercicio 5. Integración de cursores e excepcións

Desenvolve un procedemento que percorra os produtos dunha base de datos e realice unha operación sobre cada un deles.

1. Declara un cursor para seleccionar os produtos que se deben procesar.
2. Utiliza variables locais para almacenar os datos recuperados.
3. Procesa os rexistros mediante un bucle.
4. Controla o final do cursor mediante `NOT FOUND`.
5. Define un controlador para os erros SQL.
6. Comproba que o cursor se pecha correctamente ao finalizar o procedemento.
7. Realiza probas con diferentes conxuntos de datos e situacións de erro.

12. Resumo do bloque

- Os **cursores** permiten percorrer fila a fila os resultados dunha consulta SQL.
- `DECLARE CURSOR` define o cursor e a consulta asociada.
- `OPEN` abre o cursor, `FETCH` recupera unha fila e `CLOSE` péchao.
- Os cursores adoitan combinarse con variables locais e estruturas repetitivas.
- Os controladores `DECLARE HANDLER` permiten xestionar erros e condicións especiais.
- `CONTINUE` permite continuar a execución despois de tratar unha condición.
- `EXIT` permite abandonar o bloque no que foi declarado o controlador.
- `NOT FOUND` permite detectar que non quedan máis filas nun cursor.
- `SQLEXCEPTION` e `SQLWARNING` permiten capturar categorías de condicións SQL.
- `SIGNAL` permite xerar erros personalizados.
- `RESIGNAL` permite volver lanzar unha excepción capturada.
- A combinación de cursores e controladores permite desenvolver procedementos almacenados máis robustos e controlados.